Lab 1 Exercise to Do during lab hours

Student Exam Performance · Self-assessment · No solutions provided

This is a practice exercise, not a worked example. Every step tells you what to accomplish and which tools you'll likely need, you write the code yourself, following the same pattern you used in Lab 1 example (Penguins). Suggested time: 45-60 minutes.

Step 1 — Select & check the dataset

Dataset: Students Performance in Exams, published on Kaggle by user spscientist.

Link: https://www.kaggle.com/datasets/spscientist/students-performance-in-exams

Check	Result (fill this in)
Tabular, alpha-numeric?	?
Feature count	?
Enough records for a first look?	?

Your task: Open the dataset page, look at the file, and fill in the table above yourself — don't assume, check.

Step 2 — Document it

Field	Answer (fill this in)
Dataset name	Students Performance in Exams
Source	https://www.kaggle.com/datasets/spscientist/students-performance-in-exams
Access date	?
Licence	?
Original data collection	?

Kaggle itself flags this dataset as missing a stated licence and a documented data-collection source — that's not a mistake on your part, it's a real gap in the dataset's documentation.

Your task: Fill in the access date and licence rows honestly (write "not stated" if that's what you find). Then write 2-3 sentences giving your best reasoning for how this data was likely collected — for example, is it real school records, or does it look like it might be simulated/synthetic data? What in the dataset makes you think so?

Step 3 — Load it

Download the CSV from the Kaggle link above (you'll need a free Kaggle account) and place it in your project folder next to your notebook.

Your task: Write code that imports pandas, reads the CSV into a variable called df, and previews the first 5 rows.

Tools you might use:

- `import pandas as pd`
- `pd.read_csv()`
- `df.head()`

Step 4 — Describe its structure

Your task: Write code that prints the number of records, the number of features, and shows the data type of every column.

Tools you might use:

- `df.shape`
- `df.dtypes`

Remember: only the last line of a cell auto-displays — wrap earlier lines in print() if you want to see their output too.

Step 5 — Inspect data quality

Check this dataset the same way you checked Penguins' data: gaps, duplicates, consistent categories, and sensible value ranges.

Your task: Write code to check: missing values per column, duplicate rows, the unique values in each categorical column (gender, race/ethnicity, parental level of education, lunch, test preparation course), and summary statistics — handled separately for the numeric score columns and the categorical columns, the way you learned to in Lab 1b after describe() gave misleading results.

Tools you might use:

- `df.isna().sum()`
- `df.duplicated().sum()`
- `df["column_name"].unique()`
- `df["column_name"].value_counts()`
- `df[numeric_cols].describe()`
- `df.describe(include="object")`

Step 6 — Figure out your assumptions

You don't have a data dictionary or a documented collection method for this dataset (see Step 2) — so some interpretation is unavoidable.

Your task: Identify at least two assumptions you're making about this data, and where possible, check them with code. For example: are all three score columns realistically bounded between 0 and 100? Are the categories in each text column spelled consistently, with no near-duplicates?

Tools you might use:

- `df[(df["math score"] < 0) | (df["math score"] > 100)]`
- `# repeat the same idea for reading score and writing score`
- `# re-use .unique() from Step 5 to check category spelling`

Step 7 — Cluster without using a label

Group students purely by their three score columns, without telling the model anything about their background.

Your task: Scale the three score columns, run KMeans with a sensible number of clusters, add the cluster assignment back onto df, and then compare your clusters against test preparation course using a crosstab — purely to see if a pattern shows up, since the clustering step itself never saw that column.

Tools you might use:

- `from sklearn.preprocessing import StandardScaler`
- `from sklearn.cluster import KMeans`
- `scaler.fit_transform()`
- `KMeans(n_clusters=...)`
- `pd.crosstab()`

Step 8 — Predict & check accuracy

Pick one categorical column to predict from the three score columns — test preparation course is a reasonable choice, since it's plausible that scores differ between students who completed it and those who didn't.

Your task: Split your data into training and test sets, train a classifier using the three score columns as features, predict on the test set, and calculate accuracy. Then compare your accuracy against the simplest possible baseline — always guessing the most common category — to judge whether your model is actually learning anything.

Tools you might use:

- `from sklearn.model_selection import train_test_split`
- `from sklearn.tree import DecisionTreeClassifier`
- `from sklearn.metrics import accuracy_score`
- `train_test_split(X, y, test_size=..., stratify=y)`
- `model.fit() / model.predict()`
- `accuracy_score()`
- `y_train.value_counts().idxmax()  # the baseline guess`

Submission checklist

1. Filled-in tables from Steps 1 and 2, including your written reasoning about how the data was likely collected
2. Working code and output for Steps 3-5
3. Your two documented assumptions from Step 6, with supporting code output
4. Cluster crosstab from Step 7 with one sentence on what you notice
5. Accuracy result from Step 8, compared against your baseline, with one sentence on whether the model is genuinely learning something